CARRINHO ARDUÍNO

USF - Universidade de São Francisco

Campus Bragança Paulista

Materiais Elétricos e Eletrônicia

Professor Wilian Lirio

Sumula

03 **Equipe**

04 **Descrição do projeto**

05 **Objetivos e metas**

06 **Materiais usados**

07 **Função dos componentes**

08 **Funcionamento do carrinho**

09 **Arquitetura do carrinho**

10 **Sistema de alimentação**

11 **Montagem do circuito**

12 Programação do ESP32

13 Controle Bluetooth

14 **Desenvolvimento e Testes**

15 **Resultados e Melhorias Futuras**

16 **Conclusão e Acesso ao Projeto**

Equipe



Júnior Matheus



Marlon Oliveira



Matheus Lambert



Natasha Maria



Peterson Fonseca

Descrição do projeto

O Carrinho Arduino é um veículo robótico controlado por smartphone via Bluetooth BLE. Utiliza o ESP32 para interpretar os comandos e controlar os motores e o servo motor, permitindo a movimentação e o direcionamento do veículo.

- Controle: Bluetooth pelo celular.
- Processamento: ESP32.
- Movimentação: Motores DC e driver TB6612FNG.
- Direção: Servo motor.
- Programação: C++ com Arduino IDE.
- Objetivo: Integrar conhecimentos de eletrônica e programação.

Objetivos e Metas

O principal objetivo do projeto é desenvolver um carrinho robótico funcional, capaz de receber comandos remotamente por meio de um smartphone. Além da construção do protótipo, busca-se aplicar conhecimentos de programação, eletrônica e comunicação sem fio, integrando diferentes componentes em um único sistema.

Meta - 1

Desenvolvimento: construir e montar um protótipo funcional.

Meta - 2

Automação: implementar o controle de movimento e direção por Bluetooth.

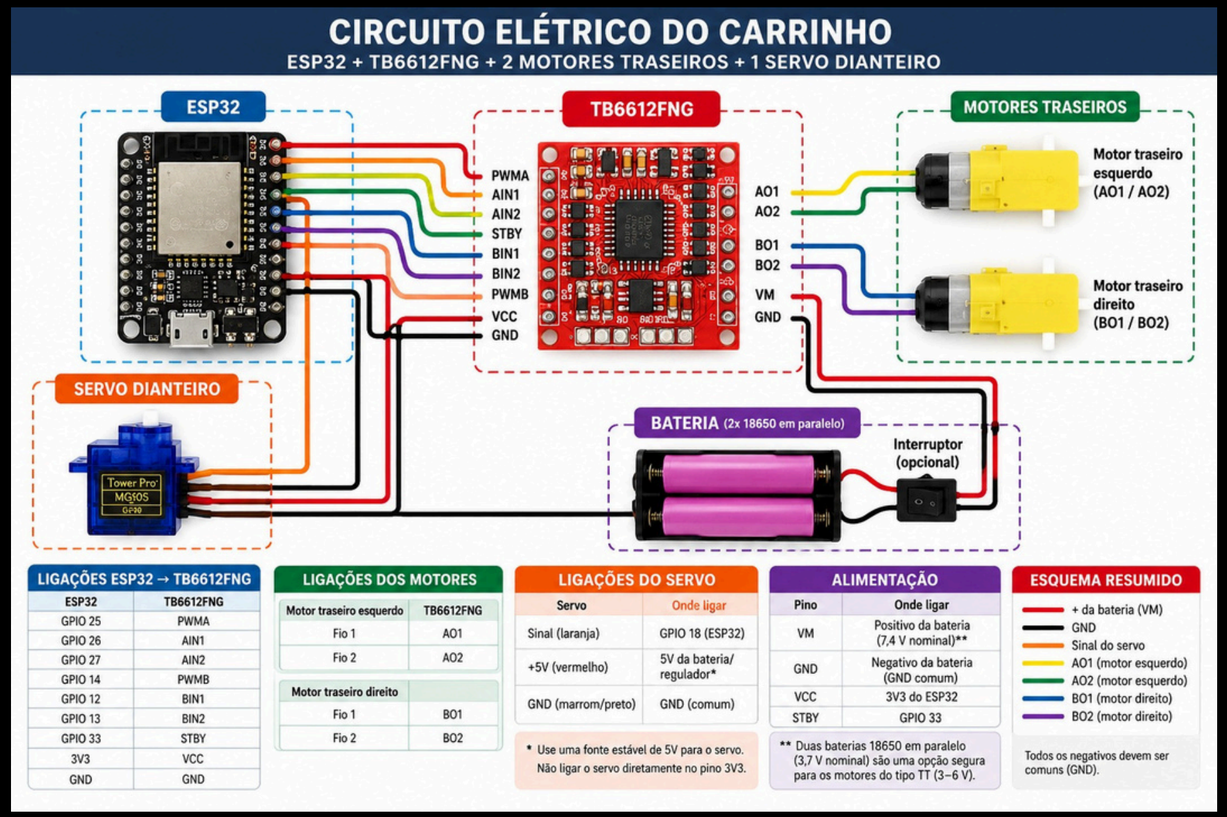
Meta - 3

Aprendizado: aplicar na prática os conhecimentos de eletrônica e programação.

Materias usados

- ESP32 DevKit V1
- Driver de motores TB6612FNG
- Servo motor S90 (6 V)
- Chassi 4W4
- Motores DC
- Duas baterias de 4 V
- Cabos de conexão (Jumper)
- Aplicativo nRF Connect
- Aplicativo BLE Controller

Imagem ilustrativa



Função dos componentes

- ESP32 DevKit V1: Responsável pelo processamento dos comandos e controle do carrinho.
- Driver TB6612FNG: Controla a velocidade e o sentido de rotação dos motores.
- Servo motor S90: Responsável pelo direcionamento do carrinho.
- Chassi 4W4: Estrutura que sustenta e organiza os componentes do veículo.
- Motores DC: Responsáveis por gerar o movimento do carrinho.
- Baterias: Fornecem energia elétrica para o funcionamento do sistema.
- Cabos Jumper: Realizam as conexões elétricas entre os componentes.
- nRF Connect: Aplicativo utilizado para estabelecer a comunicação Bluetooth BLE.
- BLE Controller: Aplicativo que permite enviar comandos pelo celular para controlar o carrinho.

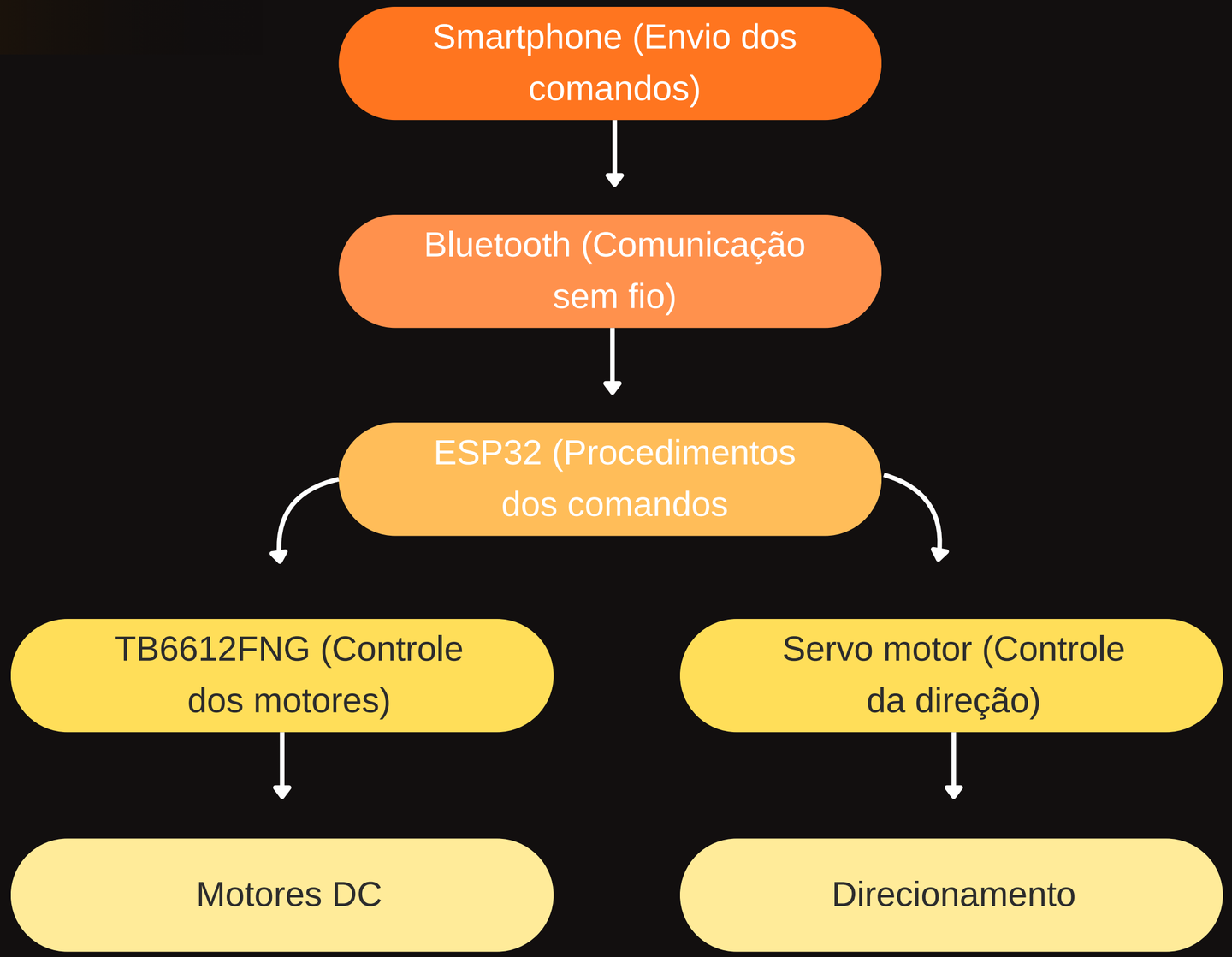
Funcionamento do carrinho

O carrinho funciona por meio de comandos enviados pelo smartphone via Bluetooth BLE. O ESP32 recebe e interpreta essas informações, acionando os motores e o servo motor conforme o comando recebido.

- Conexão: O celular se conecta ao ESP32 por Bluetooth BLE.
- Envio de comandos: O aplicativo envia instruções de movimento e direção.
- Processamento: O ESP32 interpreta os comandos recebidos.
- Movimentação: O driver TB6612FNG controla os motores, permitindo avançar, recuar e parar.
- Direcionamento: O servo motor ajusta a direção do carrinho.
- Velocidade: O usuário pode controlar a velocidade pelo aplicativo.

Arquitetura do carrinho

A arquitetura do carrinho é organizada em etapas, desde o envio dos comandos pelo celular até a execução dos movimentos. O ESP32 atua como unidade central, conectando os sistemas de comunicação, processamento e controle.

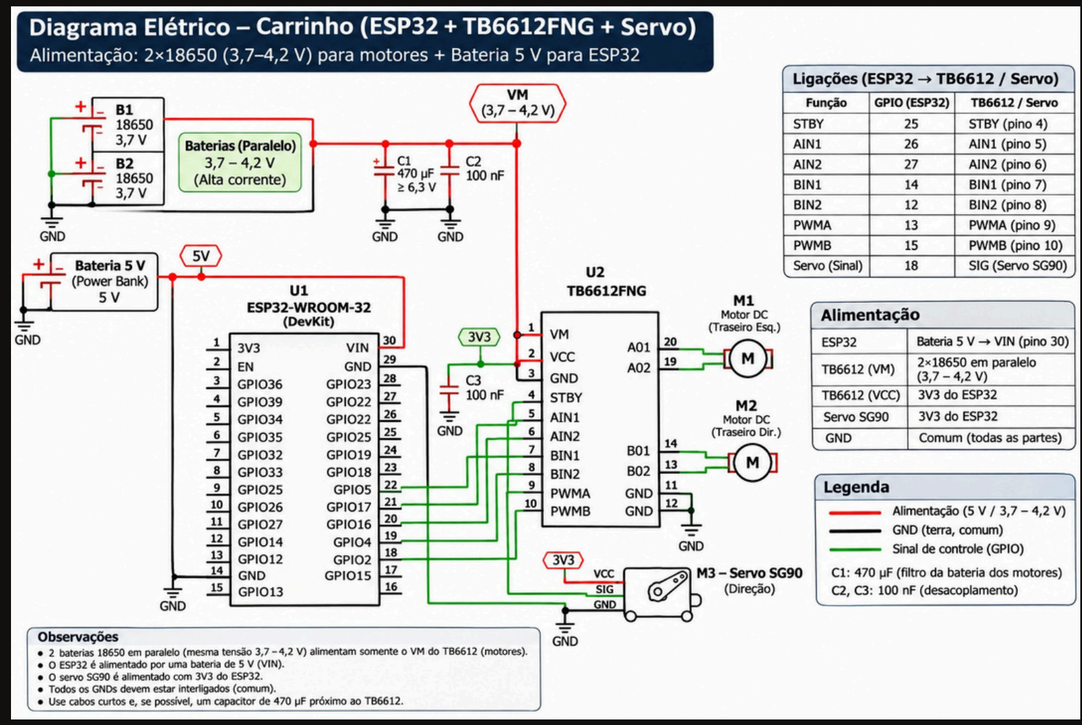


As baterias fornecem alimentação elétrica aos circuitos e motores.

Sistema de alimentação

A montagem do circuito consiste em conectar os componentes eletrônicos ao ESP32, responsável pelo controle geral do carrinho. As conexões permitem a comunicação entre o sistema Bluetooth, os motores e o servo motor.

- ESP32: Conectado ao driver e ao servo motor por meio dos pinos de controle.
- Driver TB6612FNG: Ligado ao ESP32 e aos motores DC para controlar seus movimentos.
- Motores DC: Conectados às saídas do driver para realizar a movimentação.
- Servo motor: Conectado ao ESP32 para receber os comandos de direção.
- Baterias: Integradas ao sistema para fornecer a alimentação elétrica.
- Conexões: Realizadas com cabos Jumper, seguindo o esquema elétrico do projeto.



Montagem do circuito

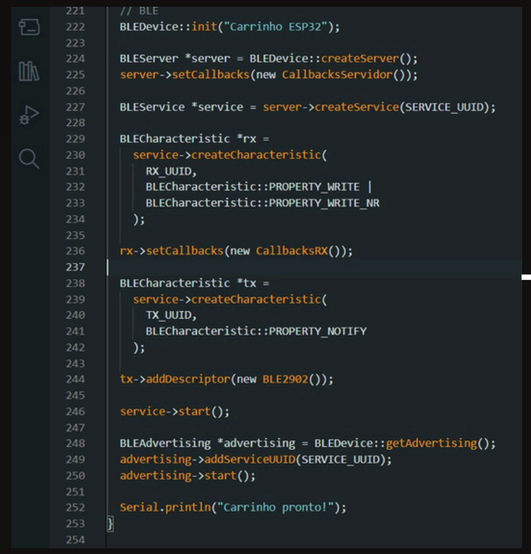
A montagem do circuito consiste em conectar os componentes eletrônicos ao ESP32, responsável pelo controle geral do carrinho. As conexões permitem a comunicação entre o sistema Bluetooth, os motores e o servo motor.

- ESP32: Conectado ao driver e ao servo motor por meio dos pinos de controle.
- Driver TB6612FNG: Ligado ao ESP32 e aos motores DC para controlar seus movimentos.
- Motores DC: Conectados às saídas do driver para realizar a movimentação.
- Servo motor: Conectado ao ESP32 para receber os comandos de direção.
- Baterias: Integradas ao sistema para fornecer a alimentação elétrica.
- Conexões: Realizadas com cabos Jumper, seguindo o esquema elétrico do projeto.

Programação do ESP32

O ESP32 foi programado em C++ pela Arduino IDE para receber comandos do smartphone via Bluetooth BLE e controlar os movimentos do carrinho.

- Conexão BLE: recebe os comandos enviados pelo aplicativo.
- Controle dos motores: determina o sentido de rotação e a velocidade dos dois motores.
- Controle da direção: ajusta o servo motor para virar à esquerda, à direita ou centralizar as rodas.
- Controle de velocidade: permite ajustar a velocidade pelo aplicativo, de 0% a 100%.
- Sistema de segurança: interrompe automaticamente os motores após 500 ms sem receber comandos.



```
// BLE
BLEDevice::init("Carrinho ESP32");

BLEServer *server = BLEDevice::createServer();
server->setCallbacks(new CallbacksServidor());

BLEService *service = server->createService(SERVICE_UUID);

BLECharacteristic *rx =
  service->createCharacteristic(
    RX_UUID,
    BLECharacteristic::PROPERTY_WRITE |
    BLECharacteristic::PROPERTY_WRITE_NR
  );

rx->setCallbacks(new CallbacksRX());

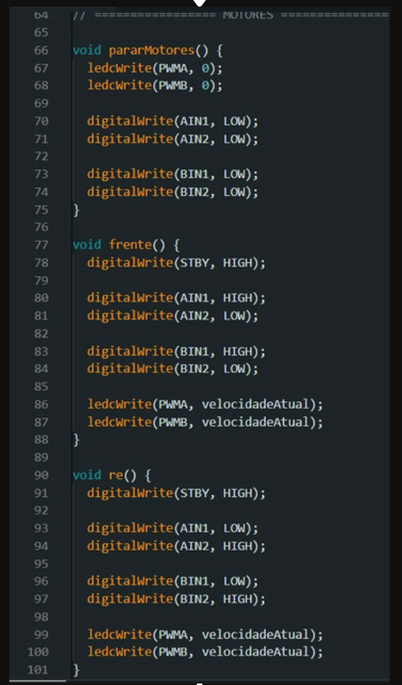
BLECharacteristic *tx =
  service->createCharacteristic(
    TX_UUID,
    BLECharacteristic::PROPERTY_NOTIFY
  );

tx->addDescriptor(new BLE2902());

service->start();

BLEAdvertising *advertising = BLEDevice::getAdvertising();
advertising->addServiceUUID(SERVICE_UUID);
advertising->start();

Serial.println("Carrinho pronto!");
}
```



```
// ================ MOTORES ================

void pararMotores() {
  ledcWrite(PWMA, 0);
  ledcWrite(PWMB, 0);

  digitalWrite(AIN1, LOW);
  digitalWrite(AIN2, LOW);

  digitalWrite(BIN1, LOW);
  digitalWrite(BIN2, LOW);
}

void frente() {
  digitalWrite(STBY, HIGH);

  digitalWrite(AIN1, HIGH);
  digitalWrite(AIN2, LOW);

  digitalWrite(BIN1, HIGH);
  digitalWrite(BIN2, LOW);

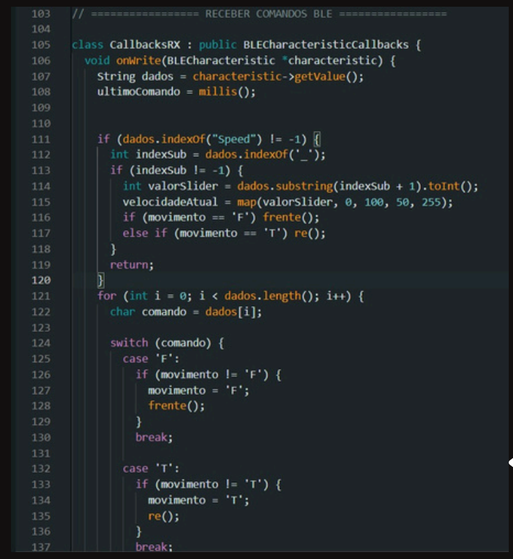
  ledcWrite(PWMA, velocidadeAtual);
  ledcWrite(PWMB, velocidadeAtual);
}

void re() {
  digitalWrite(STBY, HIGH);

  digitalWrite(AIN1, LOW);
  digitalWrite(AIN2, HIGH);

  digitalWrite(BIN1, LOW);
  digitalWrite(BIN2, HIGH);

  ledcWrite(PWMA, velocidadeAtual);
  ledcWrite(PWMB, velocidadeAtual);
}
```



```
// ================ RECEBER COMANDOS BLE ================

class CallbacksRX : public BLECharacteristicCallbacks {
  void onWrite(BLECharacteristic *characteristic) {
    String dados = characteristic->getValue();
    ultimoComando = millis();

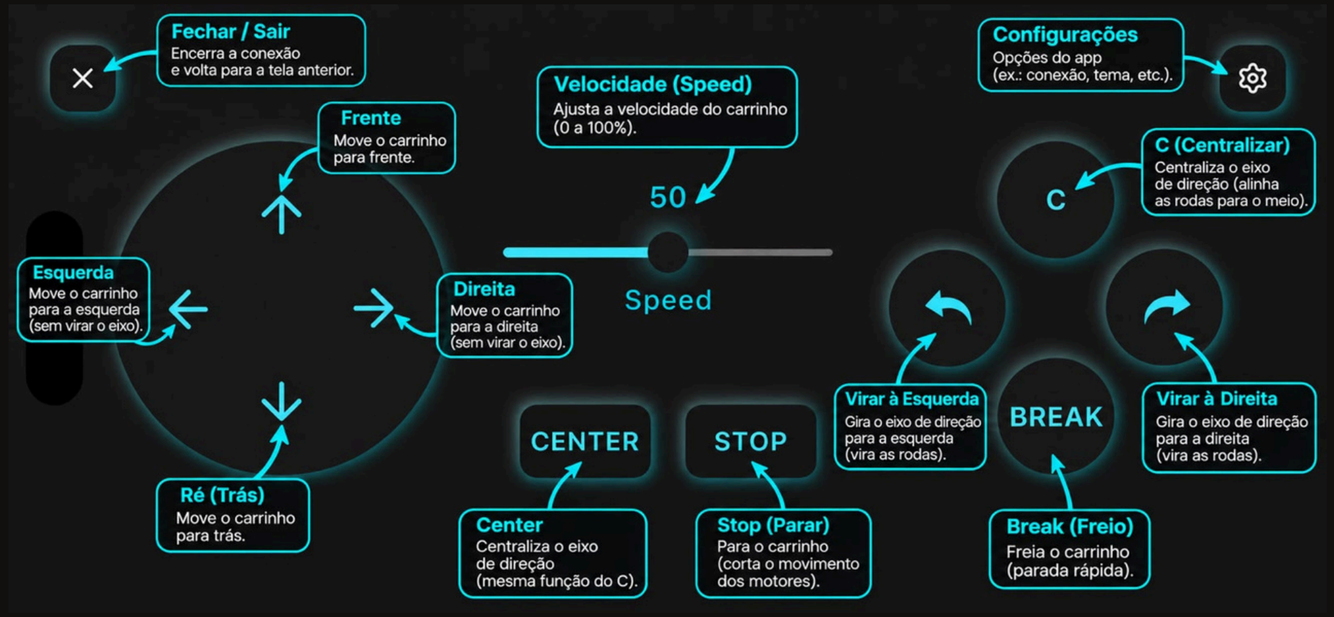

    if (dados.indexOf("Speed") != -1) {
      int indexSub = dados.indexOf('_');
      if (indexSub != -1) {
        int valorSlider = dados.substring(indexSub + 1).toInt();
        velocidadeAtual = map(valorSlider, 0, 100, 50, 255);
        if (movimento == 'F') frente();
        else if (movimento == 'T') re();
      }
      return;
    }
    for (int i = 0; i < dados.length(); i++) {
      char comando = dados[i];

      switch (comando) {
        case 'F':
          if (movimento != 'F') {
            movimento = 'F';
            frente();
          }
          break;

        case 'T':
          if (movimento != 'T') {
            movimento = 'T';
            re();
          }
          break;
```

Controle Bluetooth

O controle Bluetooth permite comandar o carrinho diretamente pelo smartphone, utilizando a tecnologia BLE para transmitir os comandos ao ESP32. Por meio do aplicativo, é possível controlar a movimentação, a direção e a velocidade, sem a necessidade de conexões físicas com o veículo.

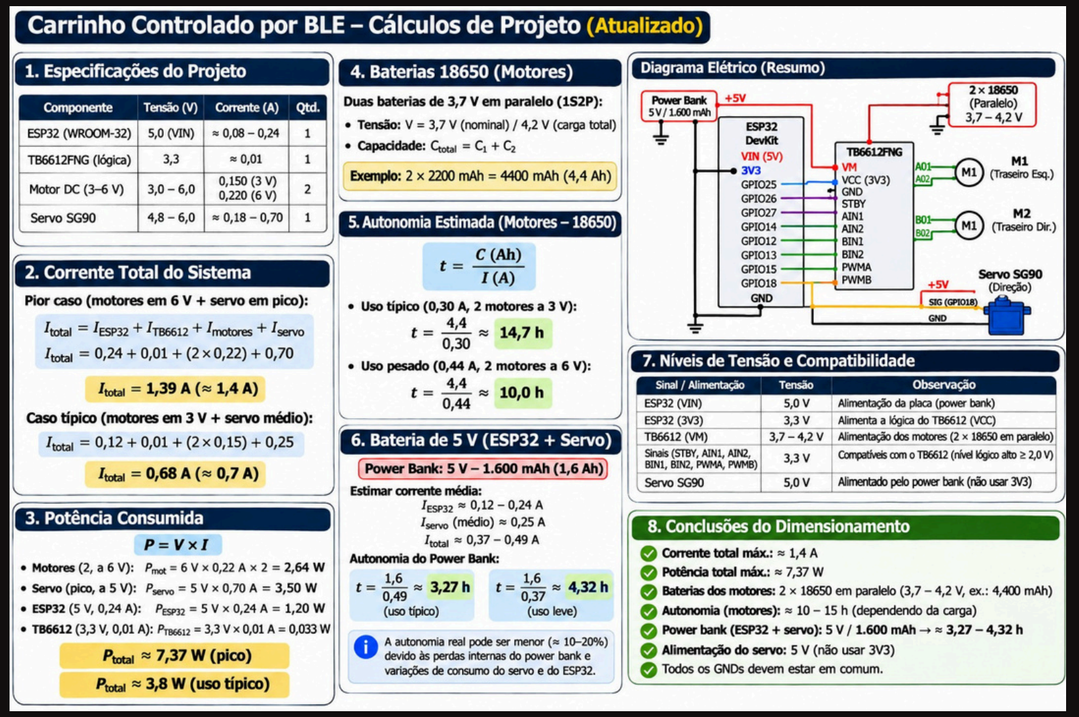


Desenvolvimento e Testes

Durante o desenvolvimento, foram realizados cálculos elétricos e análises dos componentes para definir as ligações, a alimentação e o funcionamento do circuito. Também foram feitos testes de programação, comunicação Bluetooth, movimentação dos motores e controle da direção, buscando garantir o funcionamento integrado do carrinho.

Cálculos e testes realizados:

Dimensionamento e análise da alimentação elétrica.
Verificação das conexões e dos componentes.
Testes de comunicação BLE com o smartphone.
Testes de velocidade, movimentação e direção.
Verificação do funcionamento do sistema de segurança.



Resultados e Melhorias Futuras

O desenvolvimento do carrinho permitiu integrar conhecimentos de eletrônica, programação e robótica, resultando em um protótipo funcional com controle remoto via Bluetooth. Como próximas etapas, foram planejadas melhorias para ampliar a segurança, a resistência, a autonomia e as funcionalidades do veículo.

Melhorias futuras:

Sensores de colisão: identificar obstáculos e evitar acidentes.
Monitor de bateria: acompanhar o nível de carga e indicar quando recarregar.
Reforço do chassi: aumentar a resistência e a estabilidade da estrutura.
Carroceria: melhorar a proteção e o acabamento visual.
Buzzer: adicionar alertas sonoros e sinais de funcionamento.
LEDs: implementar faróis para melhorar a visibilidade.
Kit de suspensão: proporcionar maior estabilidade em terrenos irregulares.
Reforço dos motores: melhorar a capacidade de tração e o desempenho.

Conclusão e Acesso ao Projeto

O desenvolvimento do carrinho permitiu aplicar conhecimentos de programação, eletrônica e robótica, integrando diferentes componentes em um sistema controlado por Bluetooth. O projeto também proporcionou experiência prática com o ESP32 e seus recursos.

https://petersonfonseca21.github.io/carrinho-eletronico/?utm_source=chatgpt.com

Obrigado pela atenção!